하네스 엔지니어링의 등장

하네스 엔지니어링은 AI 모델이 일을 잘하도록, 모델을 둘러싼 작업 환경(지시, 도구, 역할 분담, 작업 흐름, 검증 장치)을 설계하는 일입니다.
-> 장시간 자율 실행을 위한 역할과 작업 흐름의 설계

오케스트레이터

목표
전문 에이전트
스킬과 도구
컨텍스트 관리
검증과 피드백

01 역할을 나눈다

02 작업 흐름을 설계한다

03 상태를 이어준다

04 실수를 바로잡게 만든다

목표를 중심으로 실행과 검증이 계속 이어지는 구조

AI가 일하는 시스템을 설계하는 것

하네스 엔지니어링을 이루는 다섯 가지 요소

구성 요소	시스템에서 맡는 역할
에이전트	역할을 나누고 전문성을 부여한다
스킬 / 도구	실행에 필요한 능력을 연결한다
오케스트레이터	전체 작업 흐름을 조율한다
평가 / 테스트	품질을 검증하고 수정한다
아티팩트 / 메모리	상태를 남기고 다음 작업으로 이어간다

아키팩트 : 각 Agent가 무슨 역할을 할지, 작업 순서를 어떻게 연결할지, 어떤 도구와 데이터를 쓰게 할지를 설계하는 역할

좋은 하네스는 AI가 하나의 팀으로 일하게 한다

메타 하네스 스킬

프로젝트 설명을 실행 가능한 AI 팀 구조로 바꾸는 메타 스킬

01

입력

목표, 제약, 기술 스택

예: ir시스템 구축하고자 해.
Springboot 보안 / Next.js

02

변환

하네스 엔진이 팀 구조 설계

역할 배치와 협업 흐름
스킬 매핑

03

출력

에이전트 정의와
실행 스킬 자동 생성

즉시 실행 가능한 팀 구조

자연어로 프로젝트를 설명하면 팀 구성이 시작된다

	기본 Claude Code	메타 하네스 설치 시
파일 생성	됨	됨
진행 방식	요청한 만큼, 그때그때 판단	0~7단계 절차를 매번 동일하게
팀 패턴·실행 모드 선택	요청하지 않으면 대충 정함	기준에 따라 선택하고 이유를 남김
기존 에이전트·스킬과 중복 확인	시키지 않으면 잘 안 함	항상 확인
검증(호출 테스트, 모의 실행)	시키지 않으면 생략되기 쉬움	절차에 포함
CLAUDE.md 등록·변경 이력	시켜야 함	자동

메타 하네스 스킬 설치

마켓플레이스 추가 후 플러그인 설치

01 **마켓플레이스 추가**

/plugin marketplace add revfactory/harness

02 **플러그인 설치**

/plugin install harness@harness

대안: /plugin 실행 후 설치 메뉴에서 선택 (user scope)

영화 제작팀으로 이해하는 하네스

감독은 흐름을 조율하고, 각 전문가는 맡은 장면과 산출물에 집중한다

01	**자료 조사**	정보 수집과 검색
02	**기획·대본**	정제, 구조화, 요약과 분류
03	**촬영·편집**	분석, 추론, 생성과 계획
04	**시사·검수**	사실 검증과 품질 확인
05	**납품·기록**	포맷팅, 전달과 상태 보존



혼자 제작: 모든 작업을 담당해 병목과 확장 한계 발생

팀으로 제작: 역할 분담과 병렬 처리, 품질 일관성, 쉬운 확장

컨텍스트 관리

컨텍스트는 AI가 답을 만드는 그 순간에 볼 수 있는 모든 정보입니다.
- **시스템 지시**: AI의 기본 역할과 규칙
- **대화 내용**: 지금까지 주고받은 질문과 답
- **불러온 자료**: 읽은 파일, 검색 결과, 메모리, 스킬
- **도구 실행 결과**: 명령 실행 출력, 에러 메시지

01

작업대

지금 필요한 정보만 올린다

활성 컨텍스트

02

보관소

자주 쓰는 지식을
구조화해 저장한다

구조화된 지식

03

정리 규칙

오래된 맥락은 정리하고
핵심만 남긴다

정리와 최적화

필요한 순간에 필요한 정보를 바로 꺼내 쓰는 것이 핵심

오케스트레이터의 순차 실행

메인 에이전트(오케스트레이터)가 전체 제작 과정을 조율한다
사용자 요청 → 메인 에이전트가 오케스트레이터 스킬 로드 → 매뉴얼대로 researcher, verifier, writer 호출 → 결과 취합해서 사용자에게 보고

01	**요청 해석**	목표와 요구사항을 파악
02	**스킬·에이전트 선택**	작업에 맞는 실행 주체 결정
03	**단계별 산출물 생성**	다음 단계에서 사용할 결과 저장
04	**QA verdict 확인**	품질 판정과 리포트 확인
05	**필요 시 재실행**	검수 결과에 따라 수정

사람이 반복 지시하지 않아도 다음 작업이 이어진다

중간 산출물이 품질을 만든다

구조화된 파일을 남기면 수정과 재사용, 품질 추적이 쉬워진다

characters/{name}

sheet.json
portrait_front.png
portrait_side.png
pose_signature.png

episodes/ep{NN}

shot_list.json
prompts.json
images/cut_01.png
generation_report.json
qa_report.json
index.html

필요한 부분만 수정하고, 결과를 재사용하며, 리포트로 품질을 추적한다

좋은 하네스의 조건

표준화와 확장성, 운영성을 갖춘 지속 가능한 시스템

01

표준화

작업 방식이
흔들리지 않는다

적용: 역할 분리

02

확장성

팀과 도메인, 작업량을
쉽게 확장한다

적용: 재사용 자산화

03

운영성

검수와 재실행,
추적이 간단하다

적용: 단계 검수

하네스 엔지니어링의 핵심 3요소

에이전트, 스킬, 오케스트레이터

01 **에이전트**
역할을 가진 실행 주체

02 **스킬**
에이전트가 사용하는 능력

03 **오케스트레이터**
전체 흐름을 조율하는 두뇌

세 요소를 갖추면 멀티 에이전트 협업 구조가 시작된다

Claude Code에서 에이전트란?

역할, 입력, 출력, 도구, 규칙으로 정의하는 실행 주체
-> 에이전트는 **목표를 받으면 스스로 도구를 쓰면서 반복 작업하는 구조**입니다.

정의 항목	확인할 질문	리서치 에이전트 예시
역할	무슨 일을 맡는가	주제 조사, 정보 탐색, 자료 정리
입력	무엇을 받아 시작하는가	질문, 문서, 요구사항
출력	어떤 결과를 내야 하는가	조사 결과, 근거, 다음 액션
도구	어떤 능력과 도구를 쓰는가	검색, 분석, 리뷰
규칙	어디까지 하고 무엇을 하지 않는가	범위 준수, 출처 명확화

---
name: researcher
description: 주제에 대한 문헌·자료 조사. 논문, 기술 문서, 선행 연구를 찾고 근거와 함께 정리할 때 호출
tools: WebSearch, WebFetch, Read, Write # 도구: 조사와 결과 저장만 허용
model: opus   # 깊은 분석이 필요하므로

역할
너는 학술 리서치 담당이다. 주어진 주제의 선행 연구를 찾고, 근거가 확인된 내용만 정리한다.

입력
- 조사 주제와 범위 (호출 시 프롬프트로 전달됨)
- 있으면: _workspace/00_plan.md 의 조사 질문 목록

출력
- 파일: _workspace/01_researcher_findings.md
- 형식: 질문별로 [핵심 내용 / 출처(저자, 연도, 링크) / 신뢰도(상·중·하)]
- 마지막에 "찾지 못한 것" 섹션 필수
- 호출한 쪽에는 5줄 이내 요약만 반환

규칙
- 출처를 직접 열어 확인한 내용만 쓴다. 검색 요약문만 보고 쓰지 않는다.
  (이유: 요약문은 원문과 다른 경우가 많아 인용 오류가 생긴다)
- 2020년 이후 자료를 우선하되, 고전적 기반 논문은 연도와 무관하게 포함한다.
- 상충하는 결과가 있으면 지우지 말고 양쪽을 모두 출처와 함께 남긴다.
- 인용 형식과 정리 절차는 literature-review 스킬을 따른다.

에이전트를 만드는 5단계

작게 정의하고, 명확하게 연결하고, 반복적으로 개선한다

01	**역할 정의**	목적을 한 문장으로 정리
02	**책임 범위 설정**	할 일과 하지 않을 일 구분
03	**입력·출력 설계**	시작 조건과 결과 형식 명시
04	**스킬 연결**	필요한 도구와 능력 조합
05	**테스트와 개선**	작게 실행하고 규칙 다듬기

역할은 작게, 출력은 구체적으로, 겹치는 책임은 줄이기

스킬이 필요한 이유

에이전트의 능력을 확장하는 재사용 가능한 단위
-> **스킬은 특정 종류의 작업을 잘 수행하는 방법을 담은 폴더**입니다.

: ppt스킬, pdf 파싱 스킬

스킬이 없을 때	스킬이 있을 때
설명이 많아진다	같은 방식으로 안정적으로 실행한다
작업 방식이 흔들린다	정확도와 일관성이 높아진다
정확도와 속도가 불안정하다	스킬을 조합해 더 큰 시스템을 구성한다
재사용이 어렵다	반복 작업을 빠르게 표준화한다

파일 처리, 요약, 평가를 재사용 가능한 도구로 축적한다

---
name: literature-review
description: 학술 문헌을 검색·선별·정리하는 절차. 선행 연구 조사, 관련 연구(Related Work) 작성,
  논문 비교표 작성 시 사용한다. 일반 웹 정보나 기술 블로그 조사에는 사용하지 않는다.

문헌 조사 절차

1. 검색어 확장
핵심 용어 하나로만 검색하면 같은 개념의 다른 표현을 놓친다. 검색 전에 다음을 만든다.
- 동의어와 상위·하위 개념 (예: oversampling → data augmentation, synthetic data generation)
- 약어와 원어 (예: VAE ↔ variational autoencoder)
- 방법명과 문제명을 조합 (예: "normalizing flow" + "imbalanced tabular")

2. 선별 기준
아래 순서로 판단한다.
1. 주제 적합성: 초록에서 연구 질문과 직접 관련이 있는가
2. 출처 신뢰도: references/venue-tiers.md 의 등급을 참고한다
3. 최신성: 2020년 이후 우선. 단, 많이 인용된 기반 논문은 연도와 무관하게 포함한다

3. 기록 형식
논문 하나당 다음 항목을 기록한다.
항목	내용
서지	저자, 연도, 제목, 게재처
문제	무엇을 해결하려 했나
방법	핵심 아이디어 한두 문장
실험	데이터셋, 비교 대상, 지표
한계	저자가 밝힌 한계 + 직접 파악한 한계
관련성	우리 연구와의 관계 (기반 / 비교 대상 / 차별점)

인용 표기는 references/citation-format.md 를 따른다.

스킬을 만드는 6단계

작고 명확한 도구로 분리하고, 실제 흐름에서 검증한다

01 **반복 작업 찾기**
자주 하는 일을 식별

02 **단일 책임으로 분리**
한 가지 기능에 집중

03 **입력 정의**
필요한 값과 형식 명시

04 **출력 정의**
결과의 형식과 기준 명시

05 **예외 처리**
오류 상황과 대응 정의

06 **에이전트에 연결해 검증**
실제 흐름에서 테스트

작고, 명확하고, 재사용 가능한 스킬은 테스트를 거쳐 자산이 된다

오케스트레이터의 역할

에이전트 협업을 설계하고 조율하는 AI 팀의 PM

01 **구성 이해**

문제를 구조화하고 필요한 전문가 파악

02 **작업 분배**

누가 무엇을 할지 배치

03 **중간 점검**

진행 상태와 품질 확인

04 **결과 통합**

산출물을 연결하고 다음 단계 결정

오케스트레이터는 팀의 흐름 전체를 관리한다

오케스트레이터 안의 워크플로우

협업을 흐름으로 정의할 때 에이전트 팀이 강해진다

01 요청 수신

02 작업 분해

03 역할 연결

04 실행

05 중간 검수

06 다음 단계 연결

07 최종 산출물

워크플로우는 순서를 정한다
오케스트레이터는 연결을 책임진다
중간 검수는 실패 비용을 낮춘다

---
name: literature-survey
description: 연구 주제에 대한 선행 연구 조사를 계획부터 보고서까지 진행. "문헌 조사해줘", "선행 연구 정리" 요청 시 사용

워크플로우
1. 사용자와 조사 질문을 확정하고 _workspace/00_plan.md 에 저장
2. researcher 에이전트를 질문 그룹별로 병렬 호출 (서브에이전트 위임)
3. 결과 파일들을 모아 verifier 에이전트에 출처 검증 위임
4. 검증 실패 항목이 있으면 해당 질문만 researcher 재호출 (최대 2회)
5. writer 에이전트에 최종 보고서 작성 위임 → report.md

오류 처리
- 에이전트 실패 시 1회 재시도, 또 실패하면 보고서에 누락 사실을 명시

생성되는 핵심 산출물

역할별 전문가와 실행 도구를 함께 만든다

.claude/agents

역할 정의
책임 정리
협업 방식

.claude/skills

절차
규칙
실행 지식

에이전트 정의와 스킬이 함께 실행 가능한 팀 구조를 이룬다

6단계 구성 워크플로우

한 번의 요청이 구조 설계부터 검증까지 이어진다

구조 설계

01 도메인 분석
02 요구사항 구조화

실행 준비

03 에이전트 정의 생성
04 스킬 생성

품질 확인

05 조립·연결
06 검증·테스트